DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-3

ASSIGNMENT 3 – CO3

CO3 Coverage: Context-Free Grammars for English syntax; Context-free rules and trees; Sentence-level constructions; Agreement; Subcategorization; Parsing (Top down, Earley); Feature structures; Probabilistic Context-Free Grammars

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: ***05/11/2026***

Assignment Title: Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing (Total Marks – 50)

Duration: 1 week

DSA03 – Natural Language Processing

Assignment 3: Syntactic Analysis for a University Helpdesk Assistant

Assignment type: Individual
Total marks: 50
Student: Udhayakumar U
Reg. No.: 192424332
Roll No.: 36

The following answers are based on your attached assignment scenario. The project designs a grammar-based NLP parser for university helpdesk questions about examinations, timetables, hostels, fees, library services and placements.

Important: The assignment asks for a 30-sentence test set, at least 20 annotated parse trees, exact Earley chart entries and measured parser results. Those experimental results must be generated by running the implementation. I distinguish

TASK 1 – Design a Context-Free Grammar for English Syntax (5 Marks)

1.1 Definition of Context-Free Grammar

A Context-Free Grammar (CFG) is a formal system used to describe how words combine to form phrases and sentences.

A CFG is represented as:

$$G = (N, \Sigma, R, S)$$

Where:

- N: Set of non-terminal symbols.
- Σ: Set of terminal symbols (words).
- R: Set of production rules.
- S: Start symbol, usually the sentence symbol.

For the university helpdesk, the start symbol is S.

1.2 Non-terminal symbols

Non-terminal	Meaning
S	Sentence
NP	Noun Phrase
VP	Verb Phrase
PP	Prepositional Phrase

Det	Determiner
N	Noun
V	Verb
Adj	Adjective
AP	Adjective Phrase
Adv	Adverb
Pron	Pronoun
Aux	Auxiliary verb
WH	Wh-word
CP	Complementiser Phrase
Conj	Conjunction

1.3 Context-Free Grammar rules

The following grammar contains more than 40 phrase-structure rules.

Sentence rules

1. S -> NP VP
2. S -> VP
3. S -> Aux NP VP
4. S -> WH NP VP
5. S -> WH Aux NP VP

6. S -> S Conj S
7. S -> CP
8. CP -> Comp S
9. CP -> WH S
10. S -> NP Aux VP

Noun phrase rules

11. NP -> Det N
12. NP -> Det Adj N
13. NP -> Det AP N
14. NP -> Pron
15. NP -> ProperN
16. NP -> NP PP
17. NP -> NP Conj NP
18. NP -> Det N CP
19. NP -> Det N RelClause
20. NP -> Det Num N

Verb phrase rules

21. VP -> V
22. VP -> V NP
23. VP -> V NP NP
24. VP -> V PP
25. VP -> V CP
26. VP -> V NP PP
27. VP -> Aux V
28. VP -> Aux V NP
29. VP -> Adv VP
30. VP -> VP PP

Prepositional phrase rules

31. PP -> P NP
32. PP -> P Pron
33. PP -> P NP PP

Adjective phrase rules

34. AP -> Adj
35. AP -> Adv Adj
36. AP -> Adj PP

Relative clause and coordination rules

37. RelClause -> RelPron VP
38. RelClause -> RelPron Aux VP
39. RelClause -> RelPron NP VP
40. NP -> Det N Conj N
41. VP -> VP Conj VP
42. AP -> AP Conj AP
43. PP -> PP Conj PP
44. S -> WH Aux S
45. CP -> Comp NP VP

These rules are a controlled grammar fragment. Agreement and verb-complement restrictions are added in Tasks 4, 5 and 8.

1.4 Lexicon

The grammar and lexicon are maintained separately. The lexicon contains the actual words that can appear in helpdesk queries.

Part of speech	Example words

Determiners	the, a, an, this, that, these, those
Nouns	student, students, timetable, exam, exams, hostel, room, fee, fees, library, book, books, receipt, form, course, result, notice, placement, seminar, portal
Verbs	show, check, submit, book, reserve, send, give, tell, find, display, apply, pay, attend, postponed, crash, available
Pronouns	I, we, you, he, she, they, me, us, him, her, them
Prepositions	in, on, at, near, with, for, from, to, of, about
Adjectives	late, current, available, pending, important, new, old, postponed
Adverbs	quickly, currently, tomorrow, today, recently
Auxiliary verbs	is, are, was, were, do, does, did, can, could, will, would
Wh-words	what, when, where, why, who, which, how
Conjunctions	and, or, but
Complementisers	that, whether, if
Relative pronouns	who, which, that

This lexicon contains more than 60 distinct word forms.

1.5 Constituency, head and phrasal category

A constituent is a group of words that functions as one unit in a sentence.

A head is the main word that determines the category of a phrase.

Examples:

1. The student submitted the form.
    - The student is an NP.
    - submitted the form is a VP.
    - student is the head of the NP.
    - submitted is the head of the VP.
2. The exam in the timetable is postponed.
    - The exam in the timetable is an NP.
    - in the timetable is a PP.
    - exam is the head noun.
3. The room near the library is available.
    - The room near the library is an NP.
    - near the library is a PP.
    - room is the head noun.

Constituency tests

- Substitution: Replace the student with he.
- Movement: Move near the library to the beginning: “Near the library, the room is available.”
- Coordination: Join two NPs: “the student and the teacher”.

1.6 Why is this grammar context-free?

Each production rule replaces one non-terminal with a sequence of terminals and/or non-terminals. The rule does not depend on the surrounding symbols.

For example:

NP -> Det N

VP -> V NP

S -> NP VP

Recursive rules such as NP -> NP PP allow phrases to grow to different lengths.

However, some natural-language constructions, such as complex cross-serial dependencies, can be difficult to represent with a simple CFG. Feature-based grammar or more advanced formalisms may be required.

Result: A controlled CFG with separate phrase rules and a domain lexicon has been designed for the university helpdesk.

TASK 2 – Context-Free Rules, Derivations and Parse Trees (3 Marks)

2.1 Sample sentences

Five sentences from the helpdesk domain are:

1. The student checks the timetable.
2. The teacher submits the form.
3. The library provides books.
4. The student pays the fee.
5. The room is available.

2.2 Leftmost and rightmost derivation

Sentence: The student checks the timetable.

Grammar rules:

S -> NP VP

NP -> Det N

VP -> V NP

Leftmost derivation

S

=> NP VP

=> Det N VP

=> The N VP

=> The student VP

=> The student V NP

=> The student checks NP

=> The student checks Det N

=> The student checks the N

=> The student checks the timetable

Rightmost derivation

S

=> NP VP

=> NP V NP

=> NP V Det N

=> NP V Det timetable

=> NP V the timetable

=> NP checks the timetable

=> Det N checks the timetable

=> The N checks the timetable

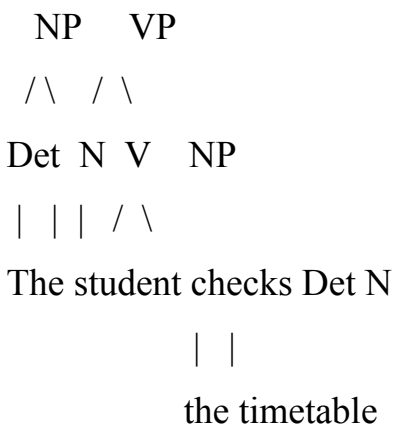
=> The student checks the timetable

2.3 Bracketed notation

```
(S
  (NP (Det The) (N student))
  (VP
    (V checks)
    (NP (Det the) (N timetable))))
```

2.4 Tree diagram

```
          S
        /   \
```



2.5 Tree measurements

For this tree:

- Number of nodes = 11.
- Height = 4 edges from root to deepest leaf.
- Number of rule applications = 5.

2.6 Ambiguous sentences

Sentence 1: “Show the students with the late fee notice.”

Two interpretations:

- The students who have the late fee notice.
- Show the students using the late fee notice.

Sentence 2: “Book the room near the library for the seminar.”

Two interpretations:

- Book the room that is near the library.
- Book the room for a seminar that is near the library.

This is called prepositional phrase attachment ambiguity.

2.7 Relationship between derivations and trees

A derivation shows the order in which grammar rules are applied. A parse tree shows the hierarchical structure of the sentence.

Different rule-expansion orders can produce the same tree. Therefore, multiple derivations may represent one syntactic structure.

2.8 Membership test

A sentence belongs to $L(G)$ if the grammar can generate the complete input string from the start symbol S.

A parser can test membership by attempting to build a complete parse tree.

Limitations include:

- Unknown words may not be in the lexicon.
- Ambiguous sentences may have multiple trees.
- Incorrect grammar rules may reject valid sentences.
- Basic CFG rules may not enforce agreement.

Result: Derivations and parse trees show how helpdesk sentences are generated and how ambiguity can occur.

TASK 3 – Sentence-Level Constructions (4 Marks)

3.1 Four main sentence types

1. Declarative sentence

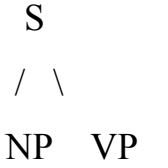
Example: “The student checks the timetable.”

S -> NP VP

NP -> Det N

VP -> V NP

Tree:



/ \ / \

Det N V NP

The student checks the timetable

2. Imperative sentence

Example: “Submit the form.”

S -> VP

VP -> V NP

The subject you is understood but is not written.

3. Yes/no question

Example: “Does the student pay the fee?”

S -> Aux NP VP

NP -> Det N

VP -> V NP

Tree:

S

/ | \

Aux NP VP

| | |

Does the student pay the fee

The auxiliary verb moves before the subject.

4. Wh-question

Example: “When is the examination?”

S -> WH Aux NP

Tree:

S

/ | \

WH Aux NP

| | |

When is examination

3.2 Subject and non-subject wh-questions

Subject wh-question:

“Who submitted the form?”

S -> WH VP

Non-subject wh-question:

“What did the student submit?”

S -> WH Aux NP VP

The missing object position is represented by a gap:

What did the student submit ___?

3.3 Subordinate clauses

Example:

“The student knows that the exam is postponed.”

S -> NP VP

VP -> V CP

CP -> Comp S

3.4 Relative clauses

Example:

“The exams that were postponed will be held tomorrow.”

NP -> Det N RelClause

RelClause -> RelPron Aux VP

3.5 Coordination

Example:

“Fees and hostel charges are pending.”

NP -> NP Conj NP

S -> NP VP

Coordination combines two or more phrases or clauses.

3.6 Effect on grammar size and ambiguity

Adding questions, relative clauses and subordinate clauses increases grammar coverage. However, it also increases the number of possible parses.
Wh-movement and long-distance dependencies become difficult to express with simple CFG rules.

Result: The grammar supports declaratives, imperatives, yes/no questions, wh-questions and subordinate constructions.

TASK 4 – Agreement (3 Marks)

4.1 Agreement phenomena

Agreement ensures that related words match grammatically.

Examples:

- Subject–verb agreement: “The student submits the form.”
- Determiner–noun agreement: “These students”, not “these student”.
- Pronoun case: “She submitted the form”, not “Her submitted the form”.

4.2 Agreement using separate categories

A basic grammar can use:

S -> NP-SG VP-SG

S -> NP-PL VP-PL

NP-SG -> Det-SG N-SG

NP-PL -> Det-PL N-PL

VP-SG -> V-SG NP

VP-PL -> V-PL NP

This prevents sentences such as:

*The students submits the form.

*The student submit the form.

4.3 Rule explosion

If we combine:

- Person: 3 values.
- Number: 2 values.
- Gender: 3 values.
- Case: 4 values.

The possible feature combinations are:

$$3 \times 2 \times 3 \times 4 = 72$$

The number of combinations grows rapidly as more features are added. Multiplying grammar categories for every combination makes the grammar difficult to maintain.

4.4 Feature-based alternative

Use features instead of creating many separate non-terminals:

NP: [NUM = SG, PER = 3]

VP: [NUM = SG, PER = 3]

The subject and verb must have matching number and person.

4.5 Test cases

Sentence	Expected result
The student submits the form.	Accept
The students submit the form.	Accept
She pays the fee.	Accept
They check the timetable.	Accept
The teacher books a room.	Accept
We attend the examination.	Accept
The students submits the form.	Reject
The student submit the form.	Reject
These student pays the fee.	Reject
Him submits the form.	Reject
She check the timetable.	Reject

They pays the fee.	Reject

A plain CFG may accept some of these incorrect sentences unless agreement is explicitly encoded.

Result: Agreement-aware grammar prevents many subject–verb, determiner–noun and pronoun-case errors.

TASK 5 – Subcategorization (4 Marks)

5.1 Definition

Subcategorization describes the type and number of complements that a verb requires.

Different verbs require different sentence structures.

5.2 Five subcategorization frames

Frame	Example	Structure
Intransitive	The portal crashed.	V
Transitive	Submit the form.	V NP
Ditransitive	Send me the receipt.	V NP NP
Sentential complement	Tell me that the fee is paid.	V NP CP
PP complement	Apply for a hostel room.	V PP

5.3 Grammar rules

VP -> V-INTR

VP -> V-TRANS NP

VP -> V-DITRANS NP NP

VP -> V-COMP NP CP

VP -> V-PP PP

5.4 Subcategorization lexicon

Verb	Frame
crash	Intransitive
submit	Transitive
check	Transitive
send	Ditransitive
give	Ditransitive
tell	Sentential complement
know	Sentential complement
apply	PP complement
pay	Transitive
reserve	Transitive

5.5 Complements and adjuncts

A complement is required or selected by the verb. An adjunct adds optional information.

Example:

- “The student submitted the form.” – the form is the complement.
- “The student submitted the form yesterday.” – yesterday is an adjunct.

5.6 Over-complemented and under-complemented sentences

Sentence	Expected result	Reason
The portal crashed.	Accept	Intransitive verb
Submit the form.	Accept	Transitive verb
Send me the receipt.	Accept	Ditransitive verb
Apply for a hostel room.	Accept	PP complement
Submit.	Reject in the required transitive frame	Object missing
The portal crashed the form.	Reject	Extra object for intransitive verb

5.7 Verbs with multiple frames

Some verbs can have multiple patterns.

Example:

- “The student paid the fee.”
- “The student paid for the room.”

A verb with multiple frames increases the number of possible parses. A subcategorization lexicon helps reduce incorrect analyses.

Result: Subcategorization rules ensure that verbs receive appropriate complements.

TASK 6 – Top-Down Parsing (4 Marks)

6.1 Definition

Top-down parsing begins with the start symbol S and expands grammar rules until the parser matches all input words.

It is also called goal-driven parsing.

6.2 Algorithm

TOP-DOWN-PARSER(sentence):

1. Start with the symbol S.
2. Select a production rule for the current non-terminal.
3. Expand the non-terminal.
4. If the next symbol is a terminal:
   Compare it with the input word.
5. If it matches:
   Move to the next word.
6. If it does not match:
   Backtrack and try another rule.
7. Continue until all words are matched.
8. If all input words are consumed:
   Accept the sentence.
9. Otherwise:
   Reject the sentence.

6.3 Example trace

Sentence: “The student checks the timetable.”

Step	Current operation
1	Start with S
2	Expand S → NP VP
3	Expand NP → Det N
4	Match The with Det
5	Match student with N
6	Expand VP → V NP
7	Match checks with V
8	Expand NP → Det N
9	Match the with Det
10	Match timetable with N
11	All words matched; accept

6.4 Ambiguous sentence

“Book the room near the library for the seminar.”

The parser may first attach near the library to the noun phrase the room. It may also try attaching a PP to the verb phrase. It must backtrack when a selected rule cannot match the remaining words.

6.5 Problems with top-down parsing

- Left recursion can cause infinite recursion.
- Repeated subtrees may be parsed many times.
- Backtracking increases execution time.
- Ambiguous sentences produce several possible analyses.

Example of left recursion:

NP -> NP PP

The parser may repeatedly call NP before consuming any input.

6.6 Left-recursion elimination

A transformed form is:

NP -> BaseNP NP-TAIL

NP-TAIL -> PP NP-TAIL

NP-TAIL -> epsilon

This allows the parser to consume the base noun phrase before adding optional PPs.

6.7 Complexity

A simple backtracking top-down parser may take exponential time in the worst case for an ambiguous grammar. With k optional PP attachments, the number of possible attachment choices can grow approximately as 2^k.

Result: Top-down parsing is easy to understand but suffers from left recursion, repeated work and ambiguity.

TASK 7 – Earley Parsing (5 Marks)

7.1 Definition

Earley parsing is a chart-parsing algorithm that uses dynamic programming to parse context-free grammars.

An Earley state contains:

- A production rule.
- A dot showing how much of the rule has been recognised.
- The origin position where the rule began.

Example:

S -> NP . VP (0)

The dot indicates that NP has been recognised and VP is expected.

7.2 Three operations

1. Predictor

When the dot is before a non-terminal, add its possible production rules to the current chart.

2. Scanner

When the dot is before a terminal, match it with the next input word and move to the next chart.

3. Completer

When a production is completed, find states waiting for that non-terminal and advance their dots.

7.3 Example sentence

“The student checks the timetable.”

Input positions:

0 The

1 student

2 checks

3 the

4 timetable

5 END

Grammar:

S -> NP VP

NP -> Det N

VP -> V NP

Det -> The | the

N -> student | timetable

V -> checks

7.4 Simplified Earley chart

The following table shows representative states for this simple grammar. A full chart implementation should preserve every state and its origin.

Chart	State	Operation
S[0]	S → • NP VP (0)	Initial
S[0]	NP → • Det N (0)	Predictor
S[0]	Det → • The (0)	Predictor

S[1]	Det → The • (0)	Scanner
S[1]	NP → Det • N (0)	Completer
S[1]	N → • student (1)	Predictor
S[2]	N → student • (1)	Scanner
S[2]	NP → Det N • (0)	Completer
S[2]	S → NP • VP (0)	Completer
S[2]	VP → • V NP (2)	Predictor
S[2]	V → • checks (2)	Predictor
S[3]	V → checks • (2)	Scanner
S[3]	VP → V • NP (2)	Completer
S[3]	NP → • Det N (3)	Predictor
S[3]	Det → • the (3)	Predictor

S[4]	Det → the • (3)	Scanner
S[4]	NP → Det • N (3)	Completer
S[4]	N → • timetable (4)	Predictor
S[5]	N → timetable • (4)	Scanner
S[5]	NP → Det N • (3)	Completer
S[5]	VP → V NP • (2)	Completer
S[5]	S → NP VP • (0)	Completer

The completed start state in the final chart confirms acceptance.

7.5 Parse tree recovery

Back-pointers record which states produced a completed constituent. They allow the parser to reconstruct the tree.

For an ambiguous sentence, the chart can share common substructures while keeping separate back-pointers for alternative PP attachments. This is called a packed parse forest.

7.6 Left recursion and ambiguity

Earley parsing handles left-recursive rules because it records states in a chart and avoids repeatedly adding identical states. It also records different attachment possibilities without requiring the entire sentence to be parsed repeatedly.

7.7 Complexity

For a general CFG:

- Worst-case time: $O(n^3)$.
- Worst-case chart space: $O(n^2)$, with grammar-dependent state factors.

Top-down parsing can repeat work. Earley parsing stores recognised states and reuses them, reducing redundant parsing.

Result: Earley parsing is more suitable than simple top-down parsing for recursive and ambiguous helpdesk sentences.

TASK 8 – Feature Structures and Unification (4 Marks)

8.1 Feature structures

Feature structures represent grammatical information using attribute–value pairs.

Example:

[CAT = NP
  NUM = SG
  PER = 3
  CASE = NOM]

8.2 Five lexical entries

student:

[CAT = N
  NUM = SG
  PER = 3]

students:

[CAT = N
  NUM = PL
  PER = 3]

submits:
[CAT = V
  NUM = SG
  PER = 3
  SUBCAT = NP]

submit:
[CAT = V
  NUM = PL
  PER = 3
  SUBCAT = NP]

she:
[CAT = PRON
  NUM = SG
  PER = 3
  CASE = NOM]

8.3 Unification

Unification combines two compatible feature structures.

Example 1:

NP: [NUM = SG]
VP: [NUM = SG]

Result:

[NUM = SG]

Example 2:

Pronoun: [CASE = NOM]

Subject: [CASE = NOM]

Result:

[CASE = NOM]

Example 3 – Failure:

NP: [NUM = PL]

VP: [NUM = SG]

The values conflict, so unification fails.

8.4 Feature-based grammar rules

S -> NP VP

where NP.NUM = VP.NUM

and NP.PER = VP.PER

NP -> Det N

where Det.NUM = N.NUM

VP -> V NP

where V.SUBCAT = TRANSITIVE

VP -> V

where V.SUBCAT = INTRANSITIVE

VP -> V PP

where V.SUBCAT = PP-COMPLEMENT

NP -> Pron

where Pron.CASE = NOM

8.5 Complete parse feature example

Sentence: “The student submits the form.”

NP:

[NUM = SG

PER = 3

CASE = NOM]

VP:

[NUM = SG

PER = 3

HEAD = submits]

S:

[NUM = SG

PER = 3]

The subject and verb agree, so unification succeeds.

8.6 Comparison

Method	Advantage	Disadvantage
Multiplied categories	Easy to understand	Many rules
Feature structures	Compact and flexible	Requires unification

Feature-based Earley	Handles constraints during parsing	More computational work

Unification can be performed during Earley's COMPLETER operation. It may increase computational cost because feature structures must be compared and merged.

Result: Feature structures provide a compact method for representing agreement and subcategorization.

TASK 9 – Probabilistic Context-Free Grammars (5 Marks)

9.1 Definition

A Probabilistic Context-Free Grammar (PCFG) assigns probabilities to grammar rules.

\[P(A\rightarrow\beta\mid A) \]

For each left-hand side non-terminal, the probabilities of all its alternative rules must add to 1.

9.2 Example PCFG

S -> NP VP 1.00

NP -> Det N 0.60
NP -> Pron 0.20
NP -> NP PP 0.20

VP -> V NP 0.50
VP -> V 0.20
VP -> VP PP 0.30

Check:

$$0.60 + 0.20 + 0.20 = 1$$

$$0.50 + 0.20 + 0.30 = 1$$

Therefore, these example rule groups satisfy the PCFG probability condition.

9.3 Maximum-likelihood estimation

The probability of a rule is:

$$P(A \rightarrow \beta) = \frac{Count(A \rightarrow \beta)}{Count(A)}$$

Example:

Rule	Count	Probability
NP → Det N	12	0.60
NP → Pron	4	0.20
NP → NP PP	4	0.20
Total	20	1.00

These counts are illustrative. The actual assignment requires counts from your own 20 annotated parse trees.

For unseen rules, smoothing can be applied by adding a small pseudocount to

each permitted rule before normalisation.

9.4 Ambiguous parse probabilities

Sentence: “Show the students with the notice.”

Two possible parses:

1. PP attaches to NP: the students have the notice.
2. PP attaches to VP: show the students using the notice.

Suppose the illustrative rule probabilities are:

$$P(NP \rightarrow NP\ PP) = 0.20$$

$$P(VP \rightarrow VP\ PP) = 0.30$$

The complete tree probability is the product of the probabilities of all rules used in that tree.

The tree with the higher product is selected as the most probable parse.

The sentence probability is:

$$P(sentence) = \sum_{T \in Parses(sentence)} P(T)$$

9.5 Viterbi CKY algorithm

CKY parsing requires a grammar converted into Chomsky Normal Form (CNF).

CNF rules have forms:

A -> B C

A -> word

Example conversion:

S -> NP VP

VP -> V NP

NP -> Det N

Viterbi CKY pseudocode:

1. Initialise a chart for all input spans.
2. Insert lexical rules for each word.
3. For spans of increasing length:
    For each split point:
        For every binary grammar rule A -> B C:
            Calculate the product of:
            P(A -> B C) × P(B) × P(C)
4. Keep the highest probability for each non-terminal.
5. Store back-pointers.
6. Return the highest-probability S parse.

9.6 PCFG limitations

- Context independence: a rule probability does not fully depend on surrounding words.
- Position invariance: the same rule probability is used wherever it occurs.
- Shorter-tree preference: trees with fewer rule applications can sometimes receive higher probabilities.
- Limited lexical sensitivity: ordinary PCFGs may not capture the meaning of specific words.

Result: PCFGs rank alternative parses using probabilities and help resolve syntactic ambiguity.

TASK 10 – Implement and Test the Parsers (3 Marks)

10.1 Python implementation

Install NLTK:

pip install nltk

10.2 Grammar and parser code

import nltkimport timegrammar_text = """S -> NP VPS -> VPNP -> Det NNP -> PronNP -> NP PPVP -> VVP -> V NPVP -> V PPPP -> P NPDet -> 'the' | 'a'N -> 'student' | 'students' | 'timetable' | 'form'N -> 'fee' | 'room' | 'library'V -> 'checks' | 'check' | 'submits' | 'submit'V -> 'books' | 'book' | 'pays' | 'pay'Pron -> 'he' | 'she' | 'they'P -> 'near' | 'for' | 'in'"""grammar = nltk.CFG.fromstring(grammar_text)def test_sentence(sentence):   words = sentence.lower().split()   # Top-down parser   start = time.perf_counter()

This is a starter implementation of the CFG, recursive-descent parser and Earley parser. It does not yet implement the complete 60-word lexicon, feature unification or PCFG. Those are extensions required for the full assignment.

10.3 Test categories

The final test set should contain:

Test category	Required examples
Declarative	The student checks the timetable.
Imperative	Submit the form.
Yes/no question	Does the student pay the fee?
Wh-question	When is the examination?
Ambiguous	Book the room near the library.

Ungrammatical	The students submits the form.
Long sentence	The student checks the timetable near the library.

10.4 Evaluation measures

Measure	Meaning
Parse found	Whether the parser accepts the sentence
Number of parses	Number of syntactic trees
Chart states	Number of states created
Execution time	Time taken to parse
Accuracy	Correct analyses divided by tested analyses

Do not report invented execution times or parse counts. Record them from the completed implementation.

10.5 Plot parser performance

After collecting measured results into a CSV file:

import pandas as pdimport matplotlib.pyplot as pltdf = pd.read_csv("parser_results.csv")for parser in df["Parser"].unique(): data = df[df["Parser"] == parser] plt.plot(data["Sentence_Length"], data["States"], marker="o", label=parser)plt.xlabel("Sentence Length (words)")plt.ylabel("Number of States")plt.title("Parser States Against Sentence Length")plt.legend()plt.grid(True)plt.show()

Result: The implementation provides a reproducible starting point for comparing top-down and Earley parsing. Complete feature and PCFG testing must be added before claiming full implementation.

TASK 11 – Compare, Recommend and Defend the Parsing Design (2 Marks)

11.1 Comparison

Feature	Top-down parser	Earley parser	PCFG parser
Parsing method	Goal-driven	Dynamic programming	Probability-based
Ambiguity	Backtracking	Stores alternatives	Ranks alternatives
Left recursion	Problematic	Handles it	Depends on parsing algorithm
Efficiency	Can repeat work	Reuses chart states	Efficient with CKY for CNF
Grammar maintenance	Simple	Moderate	Requires probabilities
Best use	Small grammar	General CFG parsing	Ambiguous sentence ranking

11.2 Recommended design

I recommend:

Feature-based CFG + Earley parser + PCFG ranking.

Reasons:

1. Feature structures handle subject–verb agreement.
2. Subcategorization rules control verb complements.
3. Earley parsing handles recursive grammar rules.
4. PCFG selects the most probable parse among alternatives.
5. The system can be extended when new helpdesk queries are introduced.

11.3 Manual effort and cost

Manual effort includes:

- Writing grammar rules.
- Preparing the domain lexicon.
- Creating and checking the 30-sentence test set.
- Annotating at least 20 parse trees.
- Testing parsers and correcting errors.
- Estimating PCFG rule probabilities.

A larger annotated corpus can reduce dependence on manually written rules, but it requires more data preparation and quality checking.

Result: A feature-based CFG with Earley parsing and PCFG ranking is the most suitable starting design for the helpdesk assistant.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

12.1 Changed requirements

The original system handled controlled and mostly grammatical sentences. The new requirement includes:

- Conversational language.
- Multi-clause sentences.
- Embedded questions.
- Relative clauses.
- Coordinated clauses.
- Missing punctuation.
- Ellipsis and sentence fragments.
- At least 20% of messages containing constructions not covered by the grammar.

12.2 Original assumptions that no longer hold

Original assumption	New situation
Controlled vocabulary	New words and informal expressions
Fully grammatical input	Missing words and grammatical errors
Simple sentences	Multi-clause messages

Limited ambiguity	More PP and clause attachment ambiguity
Small grammar	Larger grammar needed
Small test set	More diverse test data required

12.3 Impact on parsing

Longer sentences create more possible constituent combinations and more opportunities for ambiguity. A simple backtracking parser may take substantially more time. Earley parsing can reuse chart states, but the chart may still become larger as sentence length and grammar complexity increase.

B. Evaluation of the Original Design (2 Marks)

12.4 New message 1

“Can you tell me when the exams that were postponed will be held?”

The original grammar may fail because it needs:

- An embedded wh-question.
- A relative clause.
- Auxiliary constructions.
- Long-distance dependencies.

12.5 New message 2

“Hi, I need the hostel fee details and the library opening time.”

The original grammar may fail because it needs:

- A conversational greeting.
- Coordinated noun phrases.
- Multiple information requests.
- More flexible clause structures.

12.6 New message 3

“Please check my timetable, and tell me whether the examination has changed.”

The original grammar may fail because it needs:

- Imperative construction.
- Coordination of clauses.
- Embedded yes/no question.
- Auxiliary and tense handling.

12.7 Evaluation

Parser	Expected behaviour
Original top-down	May fail or require many backtracking steps
Original Earley	Handles existing CFG rules but rejects uncovered constructions
Original PCFG	Can rank only parses available in its grammar
Extended Earley	Can support new recursive and embedded constructions
Extended PCFG	Can rank new alternatives after training

These are design-based expectations, not measured test results.

C. Adaptation or Redesign (2 Marks)

12.8 Decision: Modify the original design

I recommend modifying the original system rather than completely replacing it.

The feature-based CFG and Earley parser provide a useful foundation. They can be extended with conversational rules, embedded clauses, relative clauses and partial parsing.

12.9 Revised grammar

Add rules such as:

S -> Greeting S

S -> S Conj S

VP -> V NP CP

VP -> V NP WH Aux VP

CP -> Comp S

CP -> WH S

NP -> Det N RelClause

RelClause -> RelPron Aux VP

S -> WH Aux NP VP

S -> Aux NP VP

Greeting -> 'hi'

Greeting -> 'hello'

These additions improve support for embedded questions and conversational

messages.

12.10 Feature enhancements

Add features such as:

[NUM = SG]

[NUM = PL]

[TENSE = PAST]

[TENSE = PRESENT]

[CASE = NOM]

[CASE = ACC]

[SUBCAT = TRANSITIVE]

[CLAUSE = EMBEDDED]

The feature system checks agreement, grammatical case and verb-complement requirements.

12.11 Robust parsing fallback

If a complete parse cannot be produced:

1. Tokenise the message.
2. Identify known words and phrases.
3. Find partial noun phrases and verb phrases.
4. Recover the most useful recognised structure.
5. Ask the user to clarify missing information.

Example:

User: “hostel fee tomorrow?”

System interpretation:

TOPIC = Hostel fee

TIME = Tomorrow

INTENT = Fee enquiry

The system can request clarification instead of rejecting the message.

12.12 Effect of the modifications

Factor	Expected effect
Grammar coverage	Increases
Ambiguity	May increase
Parsing cost	Increases for complex sentences
User experience	Improves
Manual annotation	More work required
Error recovery	Improves
Maintenance	Requires regular updates

D. Evidence-Based Defence (2 Marks)

12.13 Final decision

The best approach is to modify the original feature-based CFG + Earley + PCFG architecture and add a robust partial-parsing fallback.

The decision is supported by the following reasons:

1. The original CFG handles basic declarative and question structures.
2. Feature structures control agreement and subcategorization.
3. Earley parsing handles recursive and ambiguous CFG structures.
4. PCFG ranking selects the most probable parse.

5. Additional rules support embedded questions and relative clauses.
6. Partial parsing prevents the system from completely failing when it receives incomplete messages.
7. More annotated data improves probability estimation and coverage.

12.14 Remaining limitations

- Unseen words may still cause parsing failures.
- Informal messages may have incomplete grammar.
- Ambiguous sentences may have multiple valid interpretations.
- A PCFG may not understand the real-world meaning of a sentence.
- Manual annotation requires time and effort.
- The grammar must be updated as new helpdesk topics are introduced.

Final result: The modified architecture provides better coverage, ambiguity handling and error recovery for a realistic university helpdesk assistant. It is more suitable than the original small CFG alone.

Overall Conclusion

This assignment explains how context-free grammar can represent English sentence structure and how parse trees show relationships between words and phrases. Agreement and subcategorization improve grammatical correctness. Top-down parsing provides a simple parsing method, while Earley parsing uses dynamic programming to handle recursion and ambiguity. Feature structures provide an efficient method for representing grammatical constraints, and PCFGs rank alternative parse trees.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	5
Task 8 – Feature Structures and Unification	4
Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**